SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	T Deekshith	**Reg. No.:**	192472253
Section / Batch:	CSE AI / 2024	**Date:**	16/07/26

Code of Conduct

I ____T Deekshith - 192472253____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T Deekshith

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Experiment 1: Intelligent Email and Password Validator Using Regular Expressions

Aim

To develop a Python program to validate an Email Address, Password, and Mobile Number using Regular Expressions.

Algorithm

1. Import the `re` module.
2. Read the email, password, and mobile number.
3. Validate the email using the given regex pattern.
4. Validate the password according to the security rules.
5. Validate the mobile number.
6. Display the validation result.

Program

```
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
password_pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$"
mobile_pattern = r"^[6-9]\d{9}$"

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Sample Output

```
Enter Email: student123@gmail.com
Enter Password: Saveetha@123
Enter Mobile Number: 9876543210

Valid Email
Strong Password
Valid Mobile Number
```

Result

Thus, the Email, Password, and Mobile Number were successfully validated using Regular Expressions.

Experiment 2: Design a Finite State Automata (FSA) Simulator

Aim

To implement a Deterministic Finite Automata (DFA) simulator for checking whether a string ends with **“ab”**.

Algorithm

1. Define DFA states.
2. Read the input string.
3. Process each character according to DFA transitions.
4. Display the transition path.
5. Accept if the final state is reached; otherwise reject.

Program

```
transitions = {
    ('q0','a'): 'q1',
    ('q0','b'): 'q0',
    ('q1','a'): 'q1',
    ('q1','b'): 'q2',
    ('q2','a'): 'q1',
    ('q2','b'): 'q0'
}

state = 'q0'
path = [state]

string = input("Enter String: ")

for ch in string:
    state = transitions[(state, ch)]
    path.append(state)

print("Transition Path:")
print(" -> ".join(path))

if state == 'q2':
    print("Accepted")
else:
    print("Rejected")
```

Sample Output

```
Enter String: abaab

Transition Path:
q0 -> q1 -> q0 -> q1 -> q1 -> q2

Accepted
```

Result

Thus, the DFA successfully accepted strings ending with **“ab”**.

Experiment 3: Smart Pattern Matching Engine Using Regular Expressions

Aim

To develop a Python program that searches Dates, Phone Numbers, Hashtags, Mentions, Prefixes, and Suffixes using Regular Expressions.

Algorithm

1. Store the text.
2. Display the menu.
3. Read the user’s choice.
4. Perform the selected Regex search.
5. Display the matching results.

Program

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
Natural Language Processing
"""

print("1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")

choice = int(input("Enter Choice: "))

if choice == 1:
    print(re.findall(r"\d{2}/\d{2}/\d{4}", text))

elif choice == 2:
    print(re.findall(r"[6-9]\d{9}", text))

elif choice == 3:
    print(re.findall(r"#\w+", text))

elif choice == 4:
    print(re.findall(r"@\w+", text))
```

```
elif choice == 5:
    prefix = input("Enter Prefix: ")
    print(re.findall(r"\b" + prefix + r"\w*", text, re.IGNORECASE))

elif choice == 6:
    suffix = input("Enter Suffix: ")
    print(re.findall(r"\w*" + suffix + r"\b", text, re.IGNORECASE))

else:
    print("Invalid Choice")
```

Sample Output

```
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix

Enter Choice: 3

['#NLP']
```

Result

Thus, the Smart Pattern Matching Engine successfully searched various patterns using Regular Expressions.

DSA03-NLP_CO1_AT3.docx

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results

Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____